UE23CS352B - Object Oriented Analysis & Design

Mini Project Report

Title-Lost And Found System

Submitted by:

Team Members

NAME	SRN
Amrutha S	PES1UG23CS065
Anurag Sharma	PES1UG23CS085
Anusha Balamurali	PES1UG23CS086
Archi Pankaj	PES1UG23CS099

5 - B

Faculty Name-Prof. Vikram Lakshmi Kanth

January - May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

100ft Ring Road, Bengaluru – 560 085, Karnataka, India

Problem Statement:

In many public places such as colleges, offices, and transportation hubs, individuals frequently lose personal belongings. At the same time, honest finders often have no structured way to report and return found items. This lack of a centralized system leads to inefficiency, loss of valuable items, and frustration among users.

The objective of this project is to design and implement a Lost & Found Management System that enables users to report lost items, allows finders to register found items, and uses an intelligent matching mechanism to connect both parties. The system also incorporates an administrative verification process to ensure authenticity and prevent misuse.

By providing a centralized, role-based platform, the system aims to streamline the process of reporting, tracking, matching, and recovering lost items efficiently and securely.

Key Features:

Major Features (Core Functionalities)

1. Report Lost Item

    Users can submit details of lost items (category, description, location, date)

    Item is marked as “Lost”

    Automatically enters the matching system

2. Report Found Item

    Finders can register items they have found

    Includes item details and discovery location

    Item is marked as “Found”

    Requires admin verification before processing

3. Item Matching & Notification

    System matches lost and found items using:

    Sends notifications to users when a potential match is detected

    Reduces manual effort in searching

4. Claim & Verification Process

    Users can claim a matched item

    Admin verifies ownership proof

    Claim is either:

        Approved → item returned

        Rejected → item remains in system

Minor Features (Supporting Functionalities)

5. User Registration & Login

Secure authentication system

Role-based access: User or Admin

6. Search & Filter Items

Users can search items using category, date, location

Helps in quick discovery of relevant items

7. Item Status Tracking

Tracks lifecycle of an item:

Lost → Matched → Claimed → Returned

Improves transparency for users

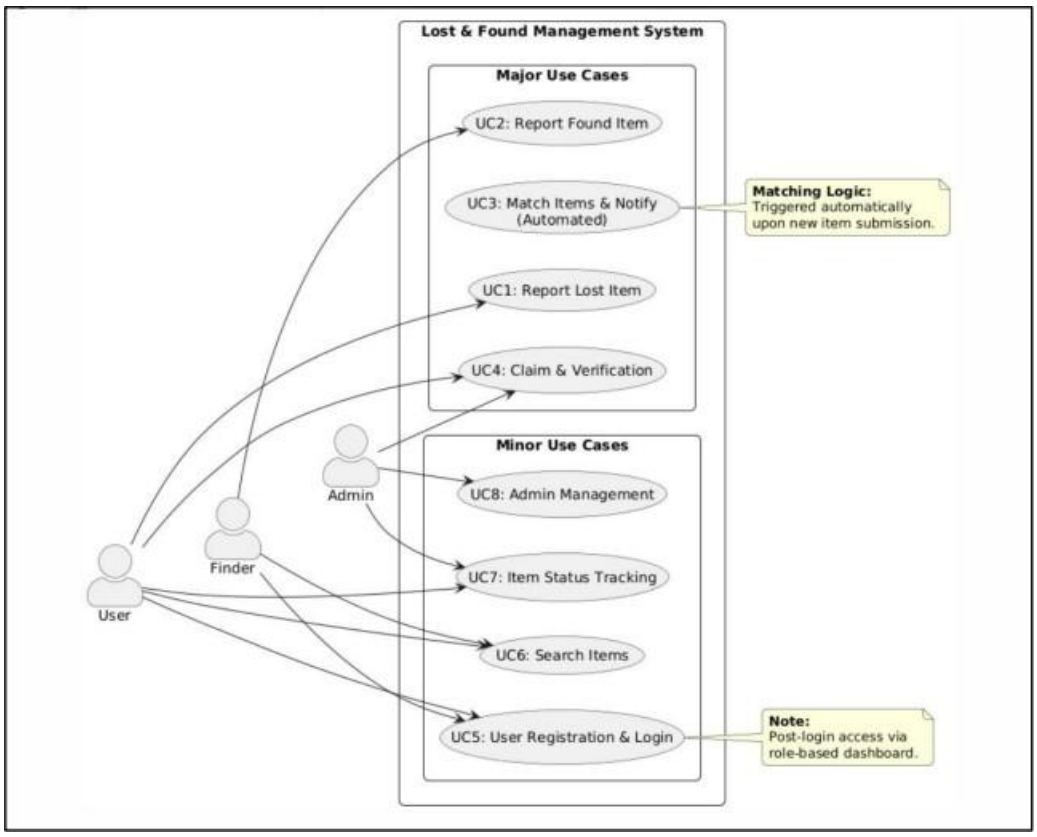
8. Admin Management

Admin can Approve/reject found items, Verify claims, Manage users

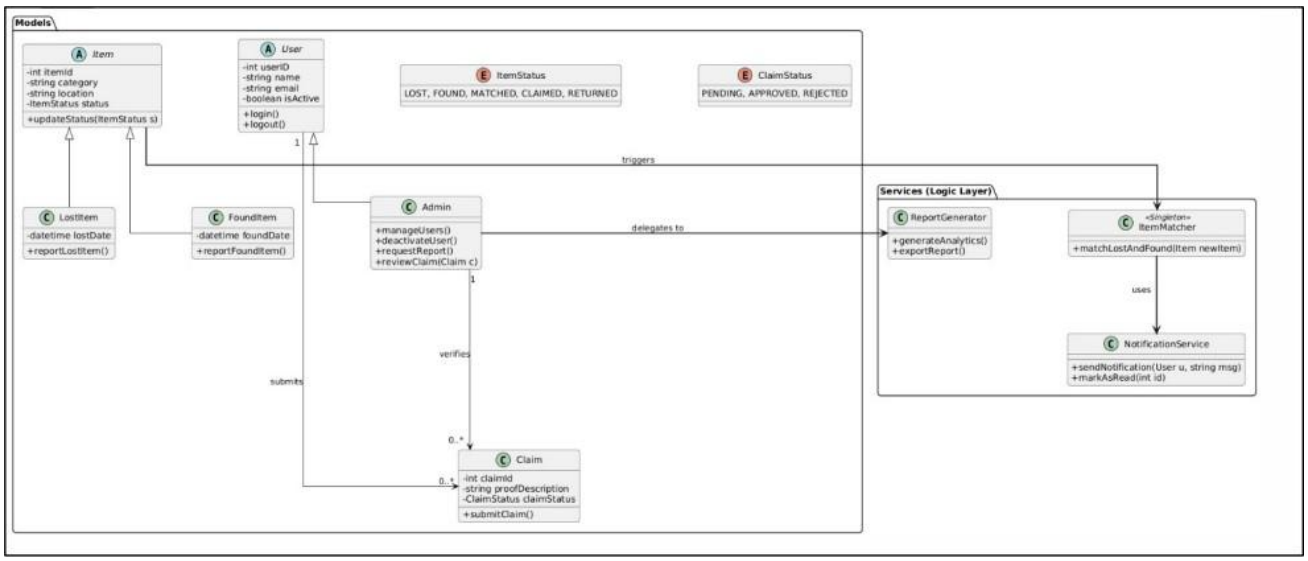
Ensures system integrity and prevents fraud

MODEL DIAGRAMS

Use Case

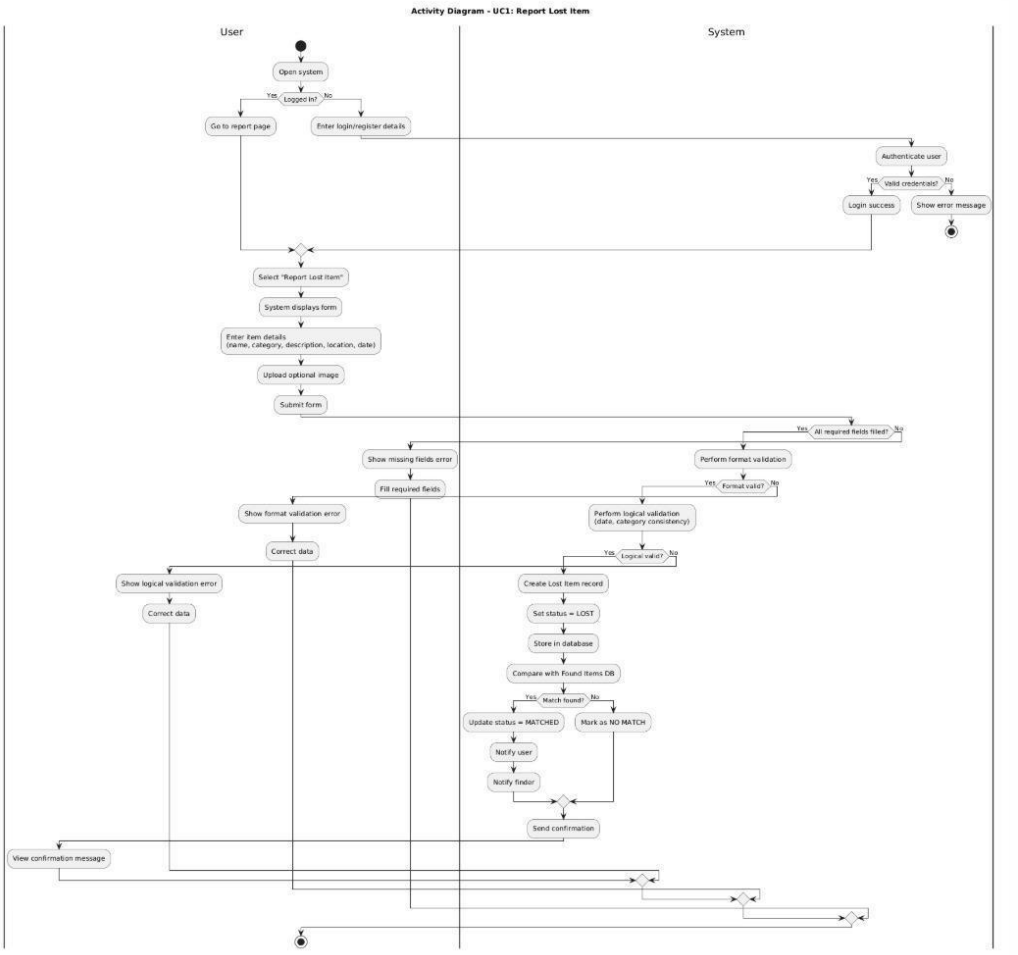


Class Diagram:

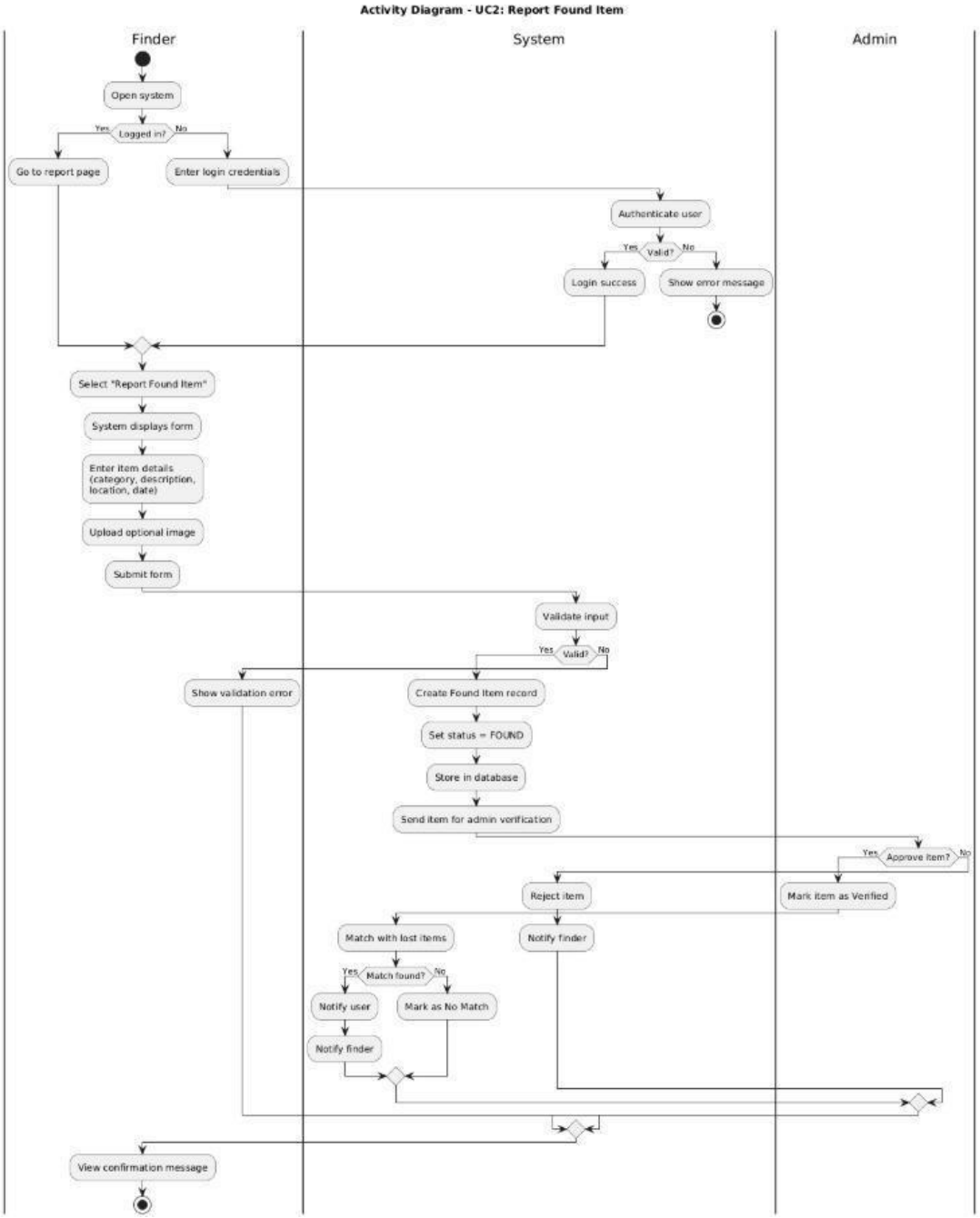


Activity Diagram:

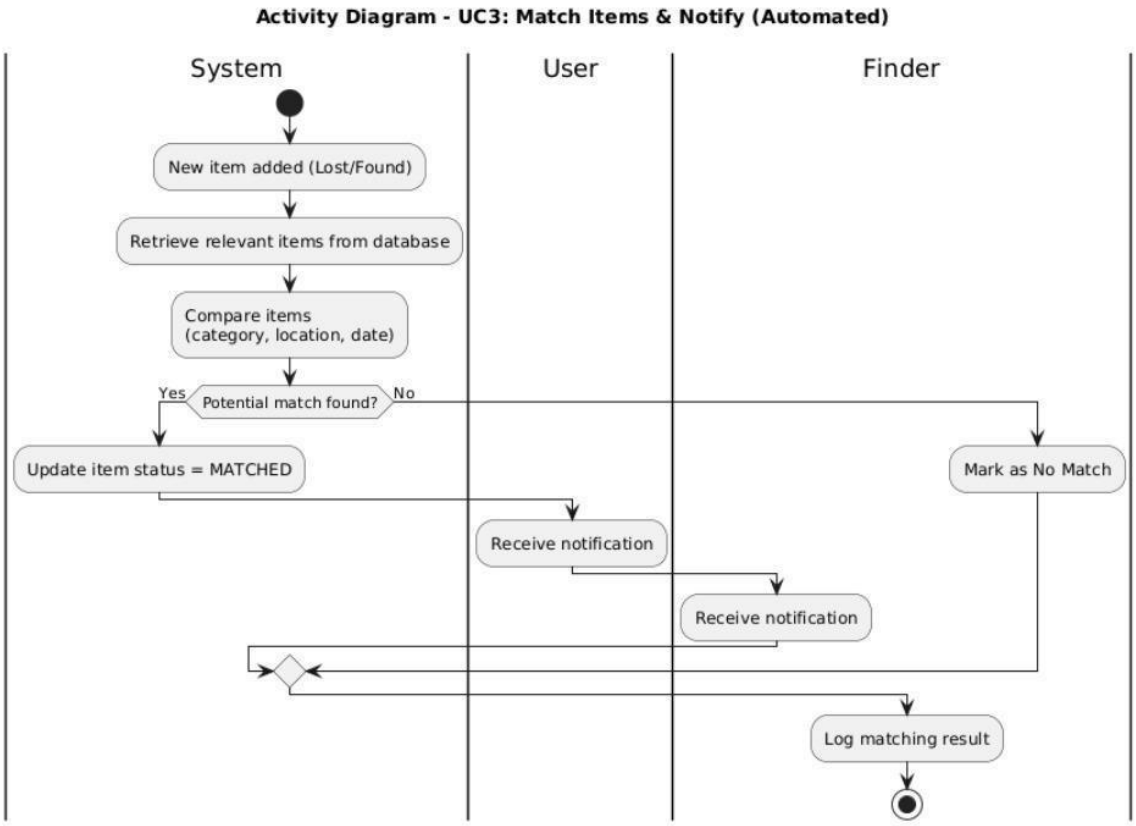
UC1: Report Lost Item



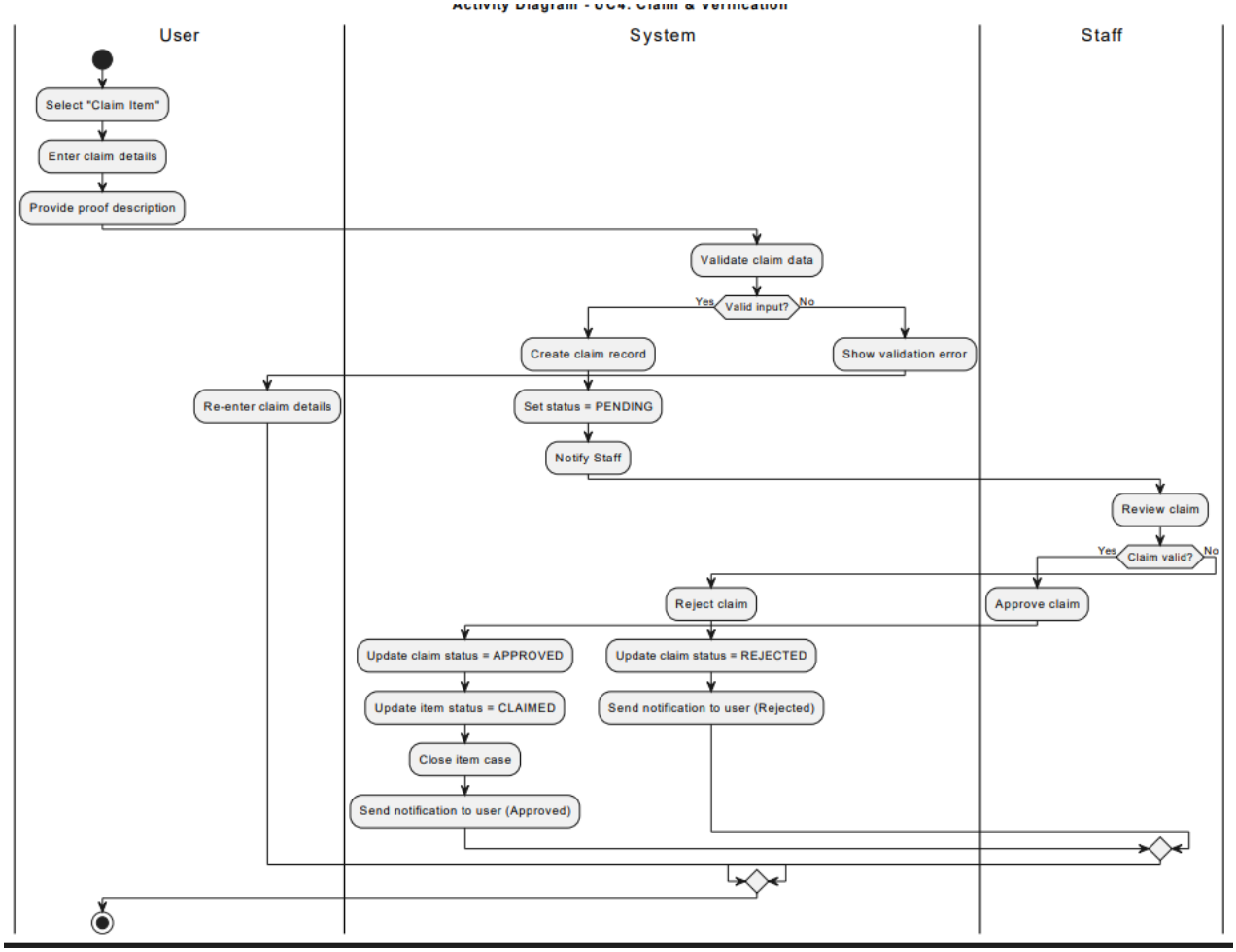
UC2: Report Found Item



UC3: Match Items & Notify (Automated)

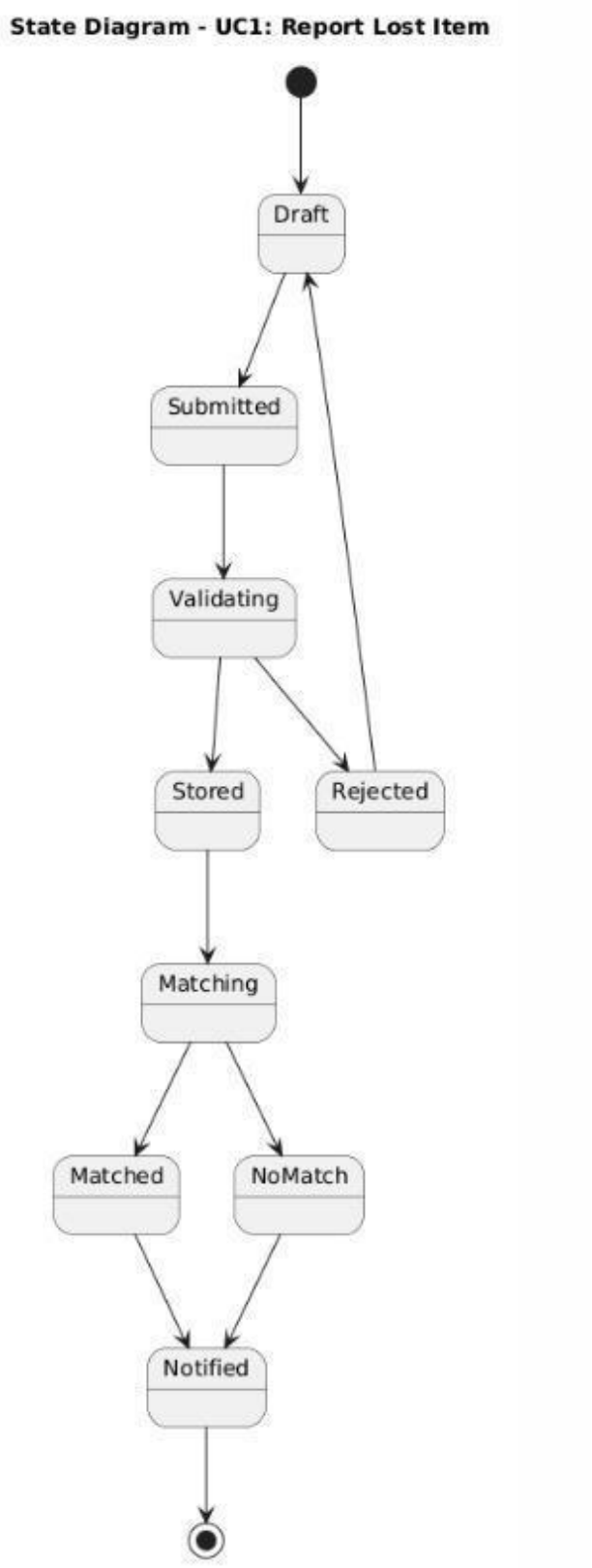


UC4: Claim & Verification

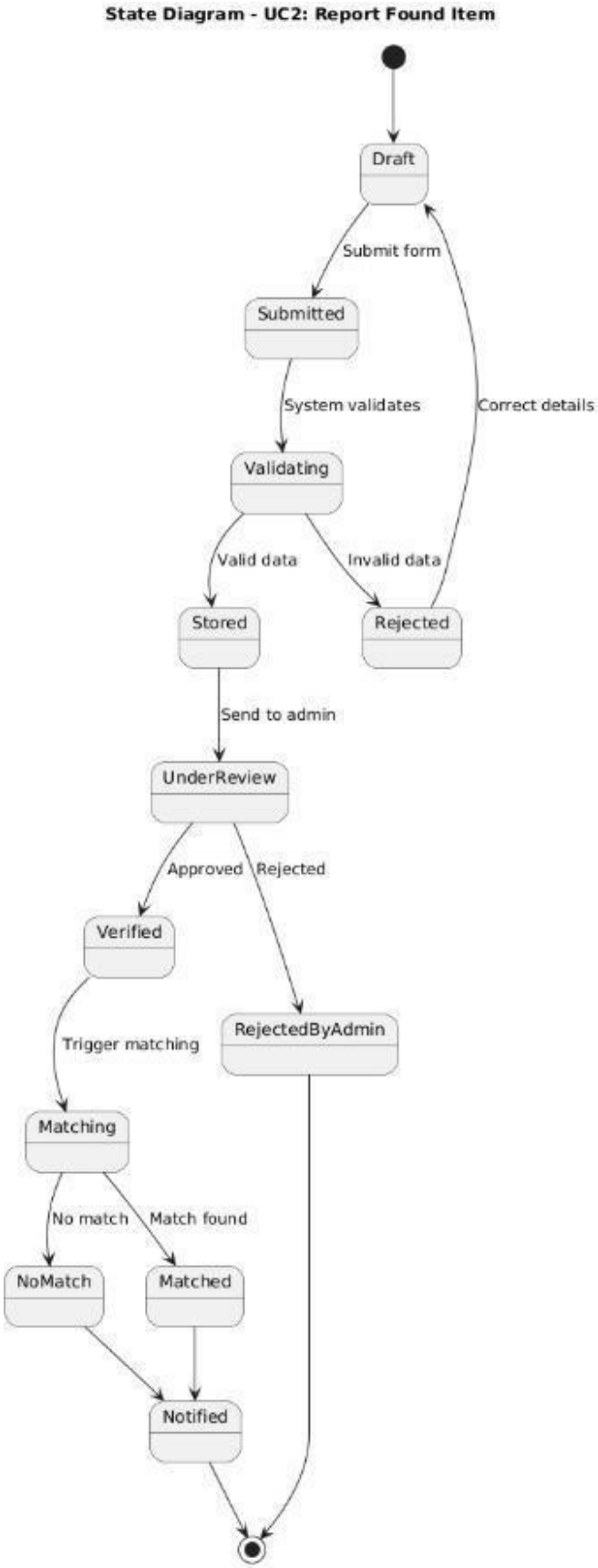


State Diagrams:

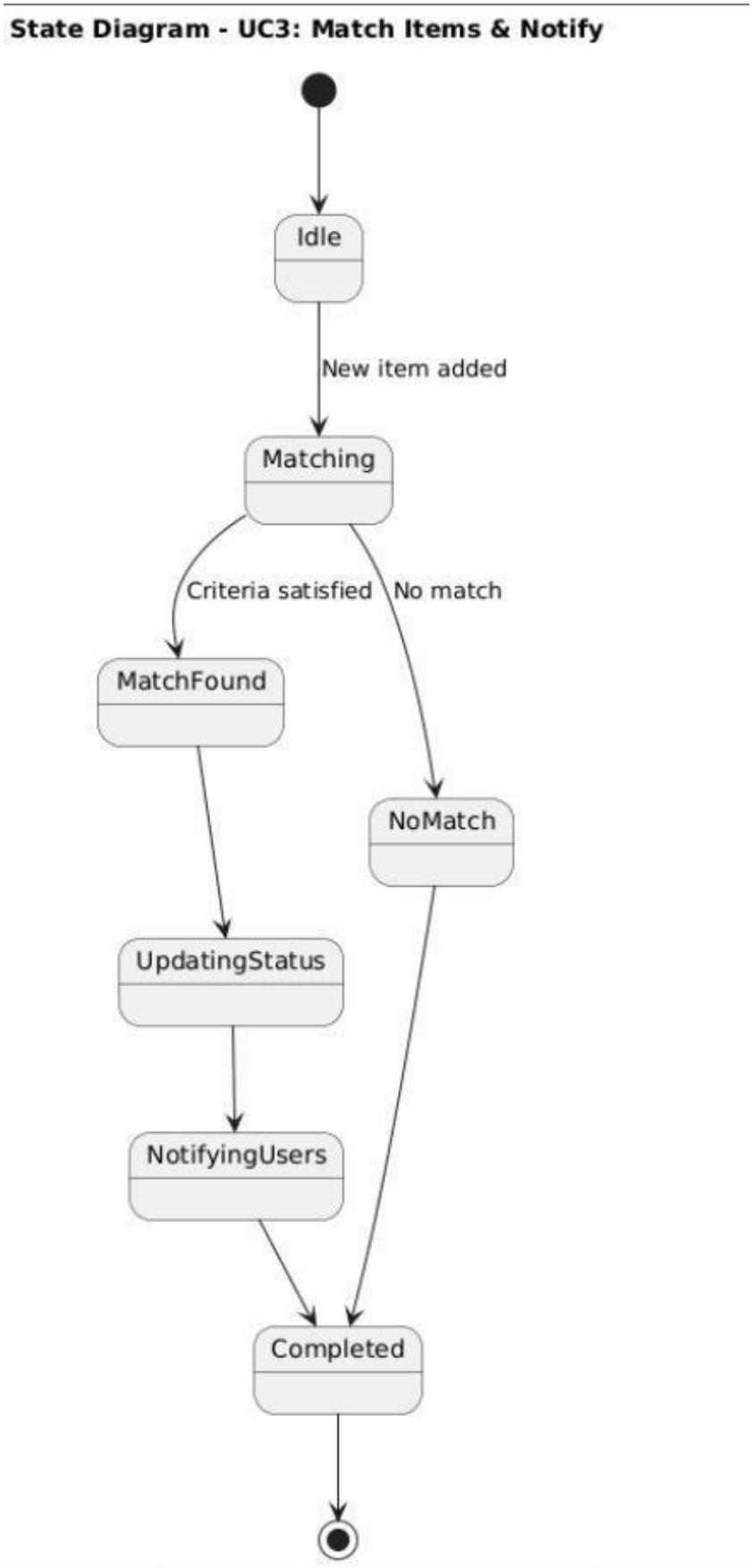
UC1:



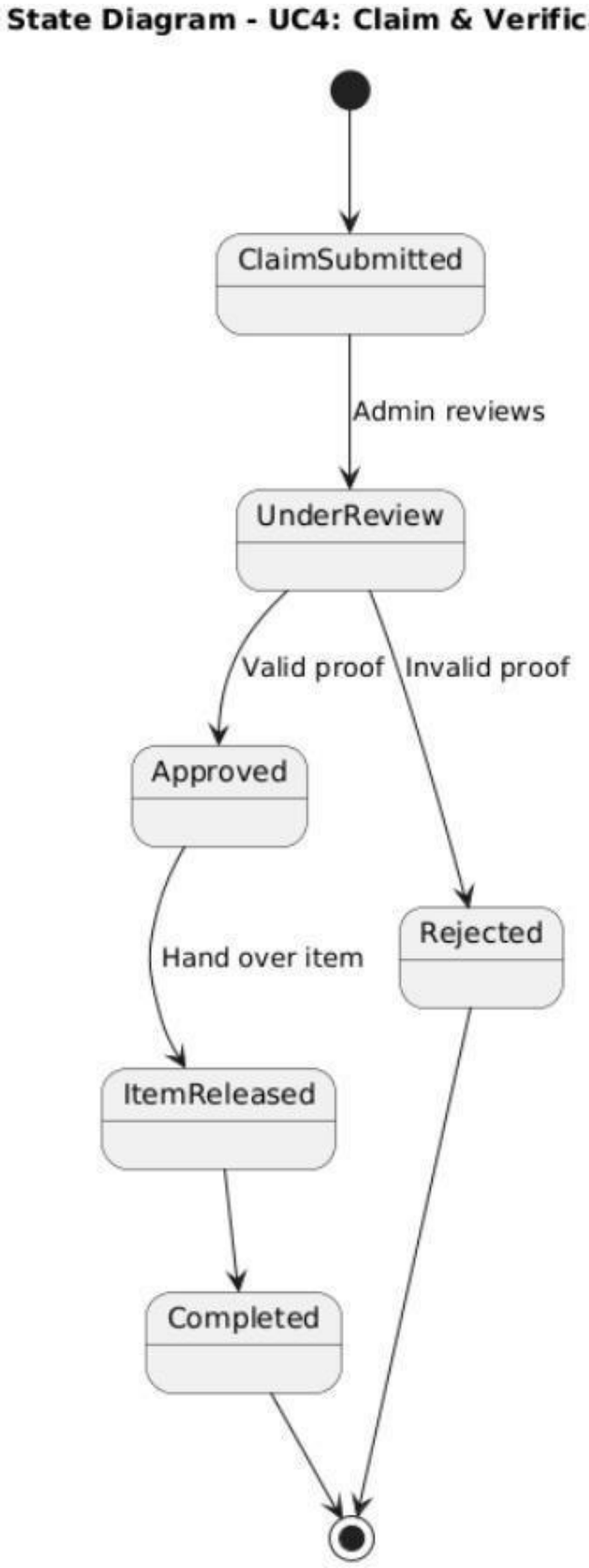
UC2:


State Diagram - UC2: Report Found Item
Draft
Submit form
Submitted
System validates
Correct details
Validating
Valid data
Invalid data
Stored
Rejected
Send to admin
UnderReview
Approved
Rejected
Verified
Trigger matching
RejectedByAdmin
Matching
No match
Match found
NoMatch
Matched
Notified

UC3:


State Diagram - UC3: Match Items & Notify
Idle
New item added
Matching
Criteria satisfied
No match
MatchFound
NoMatch
UpdatingStatus
NotifyingUsers
Completed

UC4:


State Diagram - UC4: Claim & Verification
ClaimSubmitted
Admin reviews
UnderReview
Valid proof
Invalid proof
Approved
Rejected
Hand over item
ItemReleased
Completed

Design Principles, and Design Patterns:

MVC Architecture used?
Yes

The project follows the MVC (Model-View-Controller) architectural pattern to ensure a clear separation of concerns and maintainability.

- Controller Layer:
  Handles incoming HTTP requests and user interactions.
  Examples: LostItemController, FoundItemController, MatchController, ClaimController, ViewController

- Model Layer:
  Represents the core data and business entities of the system.
  Examples: LostItem, FoundItem, Match, Claim

- Service Layer:
  Contains the business logic and processing rules of the application.
  Examples: LostItemService, FoundItemService, MatchService, ClaimService

- Repository Layer:
  Manages database operations and data persistence.
  Examples: LostItemRepository, FoundItemRepository, MatchRepository, ClaimRepository

- View Layer:
  Responsible for presenting data to the user interface.
  Example: lost-items.html rendered via ViewController

Design Principles

UC1:

SOLID Principles

- **Single Responsibility Principle (SRP):**
  Each class has a single, well-defined responsibility.
  -LostItemController handles HTTP requests
  -LostItemServiceImpl manages business logic
  -LostItemRepository handles database access
  -LostItem represents data structure

- **Dependency Inversion Principle (DIP):**
  High-level modules depend on abstractions rather than concrete implementations.
  Controllers depend on LostItemService interface instead of directly using repository classes.
- **Open/Closed Principle (OCP):**
  The system is open for extension but closed for modification.
  New business logic can be added in the service layer without modifying existing controller code.
- **Encapsulation:**
  Data hiding is maintained within model classes.
  Example: LostItem uses private fields with public getters and setters.

GRASP Principles

- **Controller:**
  LostItemController handles system operations like reporting and retrieving lost items.
- **Information Expert:**
  LostItemServiceImpl contains business rules such as setting status to "REPORTED".
- **Low Coupling:**
  Components are loosely connected.
  Controllers communicate with services, not repositories.
- **High Cohesion:**
  Each class focuses on a single task.
  All lost-item-related logic is grouped together.

UC2:

SOLID PRINCIPLES

- **Single Responsibility Principle (SRP):**
  Each component handles a specific responsibility.
  FoundItemController manages API requests.
  FoundItemServiceImpl handles business logic.
  FoundItemRepository manages database operations.
  FoundItem represents the data model.
- **Dependency Inversion Principle (DIP):**
  FoundItemController depends on the FoundItemService abstraction rather than directly interacting with the repository.
- **Encapsulation:**
  The FoundItem class maintains private fields with public getters and setters to ensure data hiding.

GRASP PRINCIPLES

- **Controller:**
  FoundItemController handles system operations such as reporting and retrieving found items.
- **Information Expert**:
  FoundItemServiceImpl is responsible for managing operations like saving and fetching found items, as it contains the required business logic.
- **Low Coupling:**
  The service layer acts as an intermediary between controller and repository, reducing direct dependencies.
- **High Cohesion**:
  All functionalities related to found items are grouped within a single module, ensuring focused responsibility.

UC3:

SOLID PRINCIPLES

- **Single Responsibility Principle (SRP):**
  Each component has a clearly defined role.
  MatchController handles API endpoints.
  MatchServiceImpl handles match creation, confirmation, and notification logic.
  MatchRepository handles database persistence.
  DTOs (MatchRequestDto, MatchResponseDto) manage data transfer between layers.
- **Dependency Inversion Principle (DIP):**
  MatchController depends on the MatchService interface rather than concrete implementations.
- **Interface Segregation Principle (ISP):**
  The service interface exposes only match-related operations, avoiding unnecessary methods.
- **Encapsulation:**
  Match, MatchRequestDto, and MatchResponseDto encapsulate their data using private fields and controlled access methods.

GRASP PRINCIPLES

- **Controller**:
  MatchController handles operations such as creating, retrieving, and confirming matches.
- **Information Expert:**
  MatchServiceImpl manages match creation, updates status, and triggers notifications as it contains the required business logic and collaborators.
- **Low Coupling:**
  Controllers interact with services, and services interact with repositories, minimizing direct dependencies.
- **High Cohesion:**
  All match-related logic is concentrated within MatchServiceImpl, ensuring focused responsibility.

UC4:

- **SOLID PRINCIPLES**

  Single Responsibility Principle (SRP):
  Each component has a specific responsibility.
  ClaimController handles claim-related API requests.
  ClaimServiceImpl manages claim creation and verification logic.
  ClaimRepository handles database persistence.
  Claim represents claim data.
- **Dependency Inversion Principle (DIP):**
  The controller depends on the ClaimService abstraction rather than directly interacting with the repository.
- **Encapsulation:**
  Claim and ClaimDTO use private fields with getters and setters to ensure data hiding.

GRASP PRINCIPLES

- **Controller:**
  ClaimController handles operations such as creating and verifying claims.
- **Information Expert:**
  ClaimServiceImpl manages claim status by setting initial status to "PENDING" and updating it to approved or rejected based on verification.
- **Low Coupling:**
  The controller does not directly interact with the database and communicates through the service layer.
- **High Cohesion**:
  All claim-related responsibilities are grouped within the claim module, including service, model, and repository.

Design Patterns

UC1:

- **MVC Pattern**:
  Separates the application into Model, View, and Controller layers.
- **Repository Pattern:**
  Abstracts data access logic.
  Example: LostItemRepository extends JpaRepository.
- **Service Layer Pattern**:
  Encapsulates business logic within service classes.
  Example: LostItemService and LostItemServiceImpl.
- **Dependency Injection Pattern**:
  Promotes loose coupling by injecting dependencies.

UC2:

- **MVC Pattern:**
  Separates the application into Model, View, and Controller components.
- **Repository Pattern:**
  Abstracts database access logic.
  Example: FoundItemRepository.
- **Service Layer Pattern:**
  Encapsulates business logic within service classes.
  Example: FoundItemService and FoundItemServiceImpl.
- **Dependency Injection**:
  Dependencies reducing tight coupling between components.

UC3:

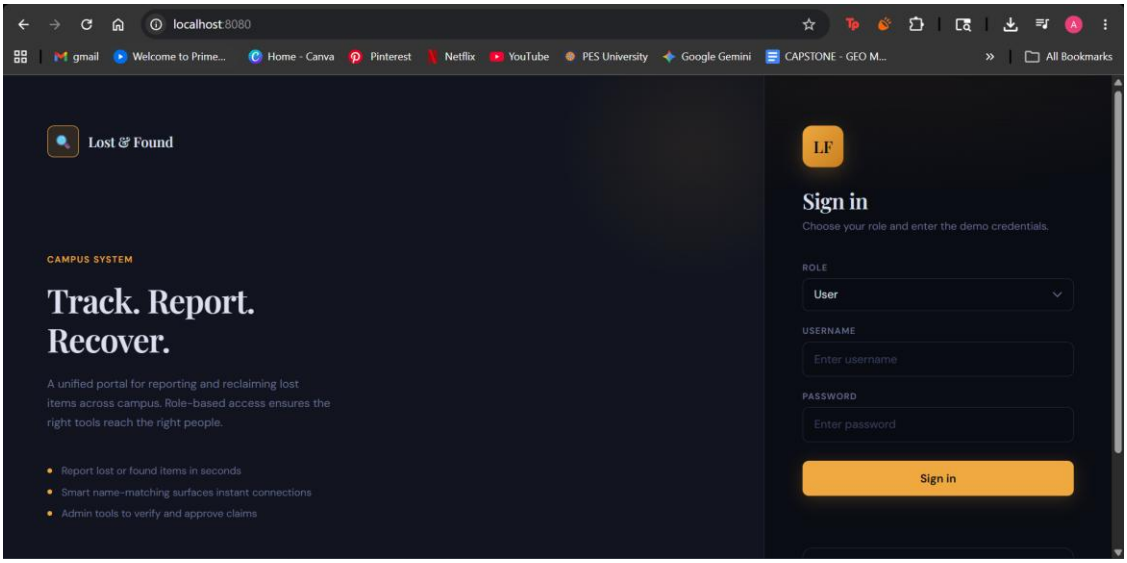
- **MVC Pattern:**
  Separates the system into Model, View, and Controller layers.
- **Repository Pattern:**
  Handles data access logic.
  Example: MatchRepository.
- **Service Layer Pattern:**
  Encapsulates business logic.
  Example: MatchService and MatchServiceImpl.
- **DTO Pattern:**
  Used for transferring data between layers.
  Examples: MatchRequestDto, MatchResponseDto.
- **Dependency Injection:**
  Dependencies to reduce tight coupling.
- **Notification Handling:**
  Notification logic is implemented within sendMatchNotification() as a direct service method. It is not a full Observer pattern but a simple internal notification mechanism.

UC4:

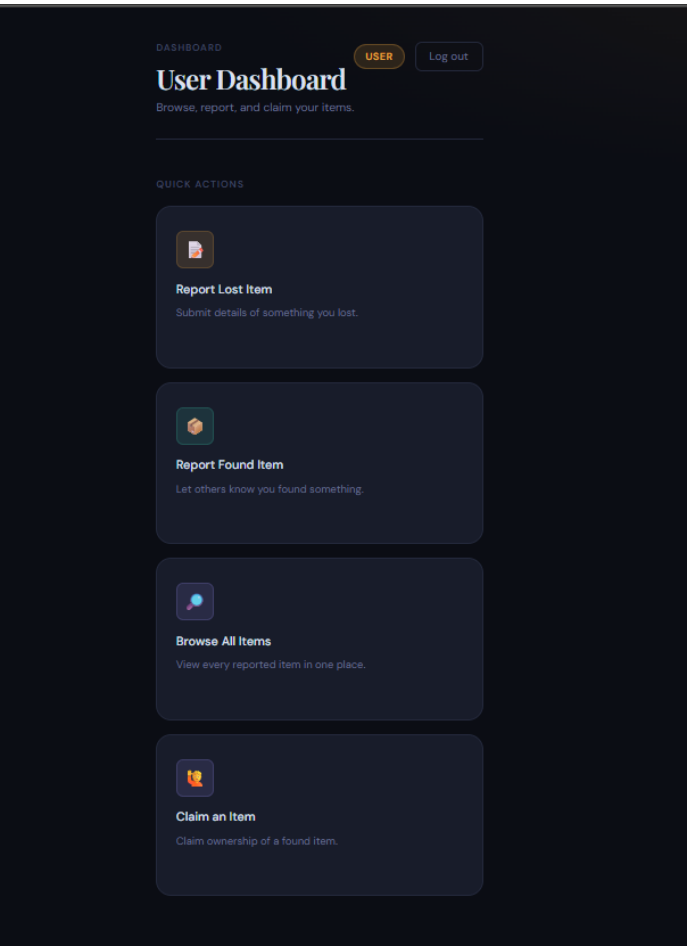
- **MVC Pattern**:
  Separates the application into Model, View, and Controller.
- **Repository Pattern:**
  Handles data access logic.
  Example: ClaimRepository.
- **Service Layer Pattern:**
  Encapsulates business logic.
  Example: ClaimService and ClaimServiceImpl.
- **DTO Pattern:**
  ClaimDTO is used for input handling, and additional DTOs like ClaimRequestDto and ClaimResponseDto may be used in extended implementations.
- **Dependency Injection:**
  Dependencies are injected to reduce tight coupling.

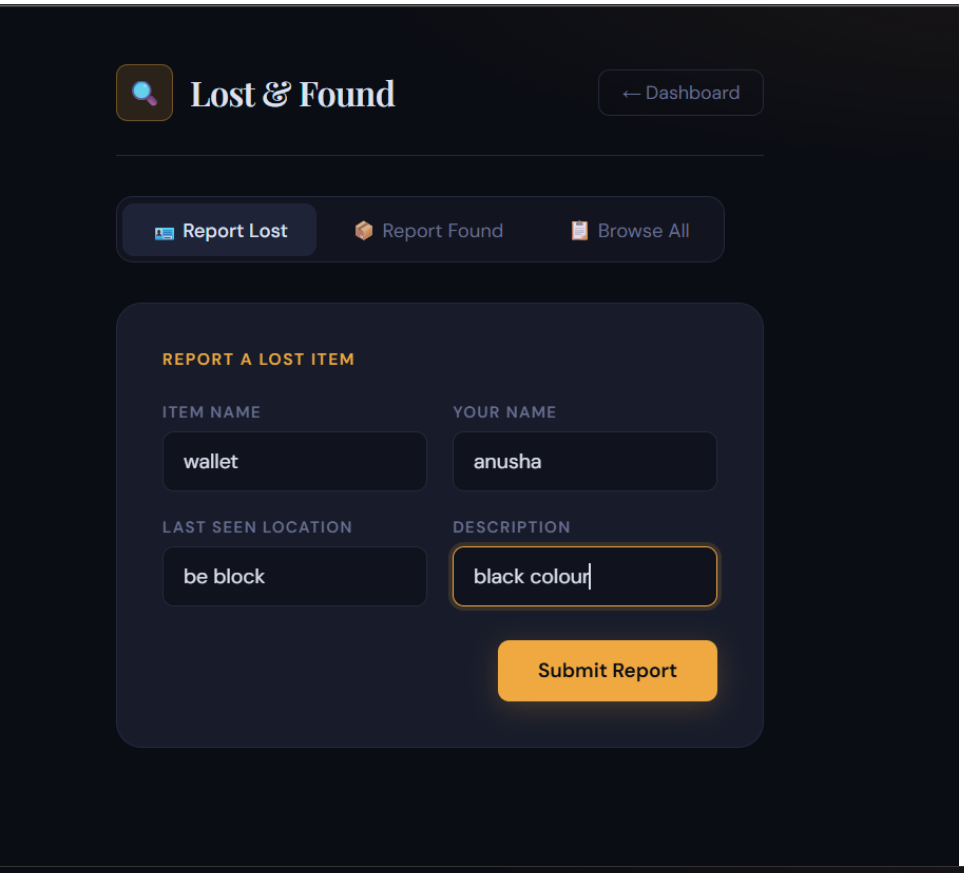
Github link to the Codebase: (repository should be public)
https://github.com/amrutha0307/Lost-and-Found

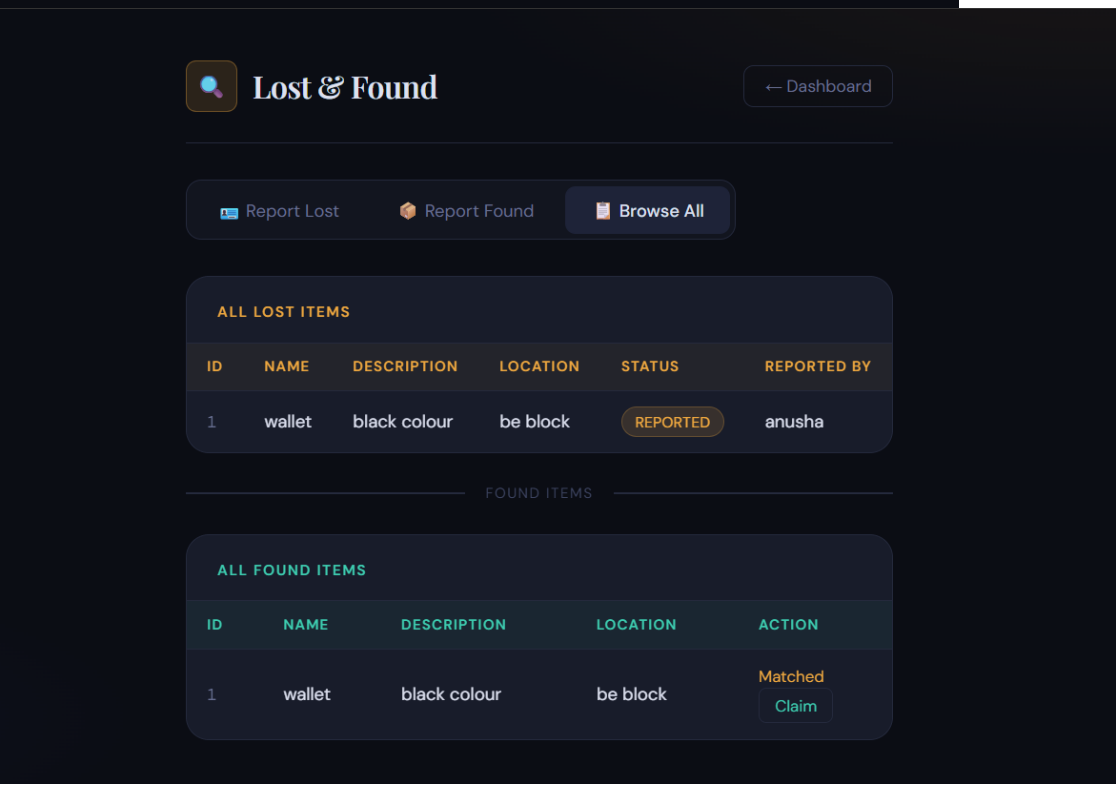
UI:

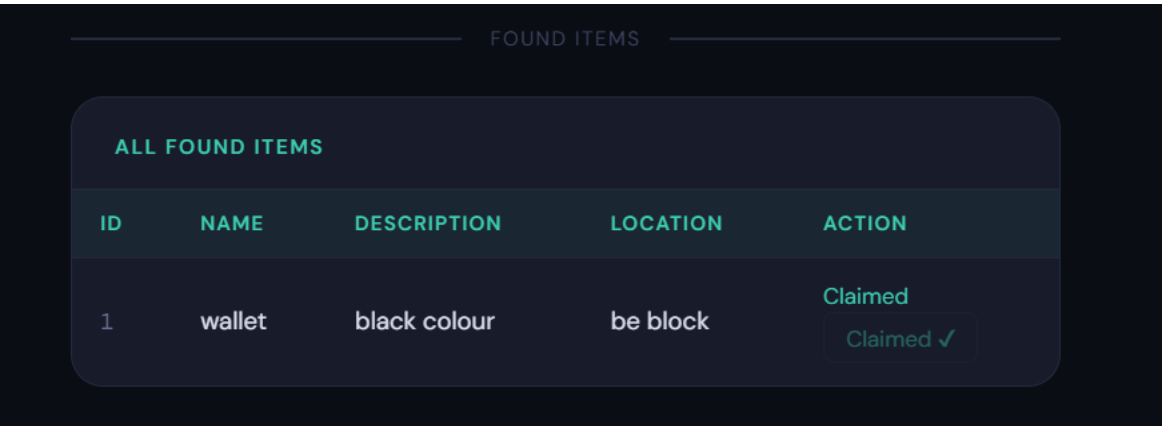

localhost:8080
gmail
Welcome to Prime...
Home - Canva
Pinterest
Netflix
YouTube
PES University
Google Gemini
CAPSTONE - GEO M...
All Bookmarks
Lost & Found
CAMPUS SYSTEM
Track. Report.
Recover.
A unified portal for reporting and reclaiming lost items across campus. Role-based access ensures the right tools reach the right people.
Report lost or found items in seconds
Smart name-matching surfaces instant connections
Admin tools to verify and approve claims
LF
Sign in
Choose your role and enter the demo credentials.
ROLE
User
USERNAME
Enter username
PASSWORD
Enter password
Sign in

User side-

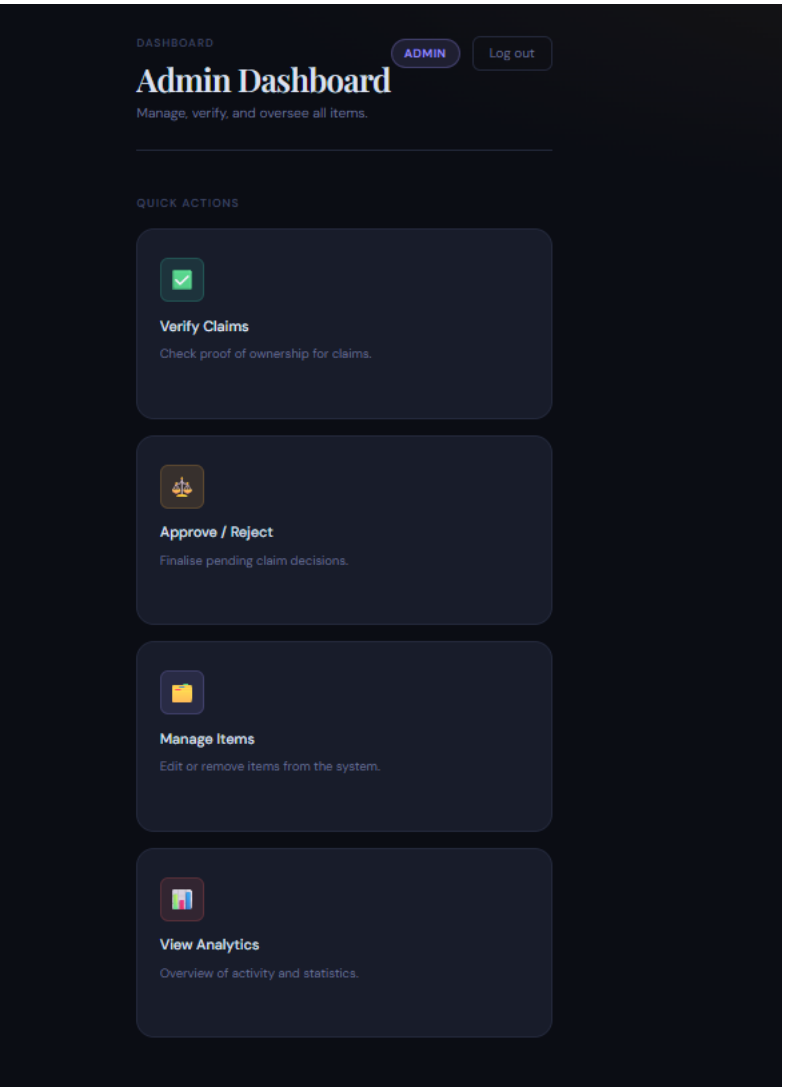

DASHBOARD
User Dashboard
USER
Log out
Browse, report, and claim your items.
QUICK ACTIONS
Report Lost Item
Submit details of something you lost.
Report Found Item
Let others know you found something.
Browse All Items
View every reported item in one place.
Claim an Item
Claim ownership of a found item.

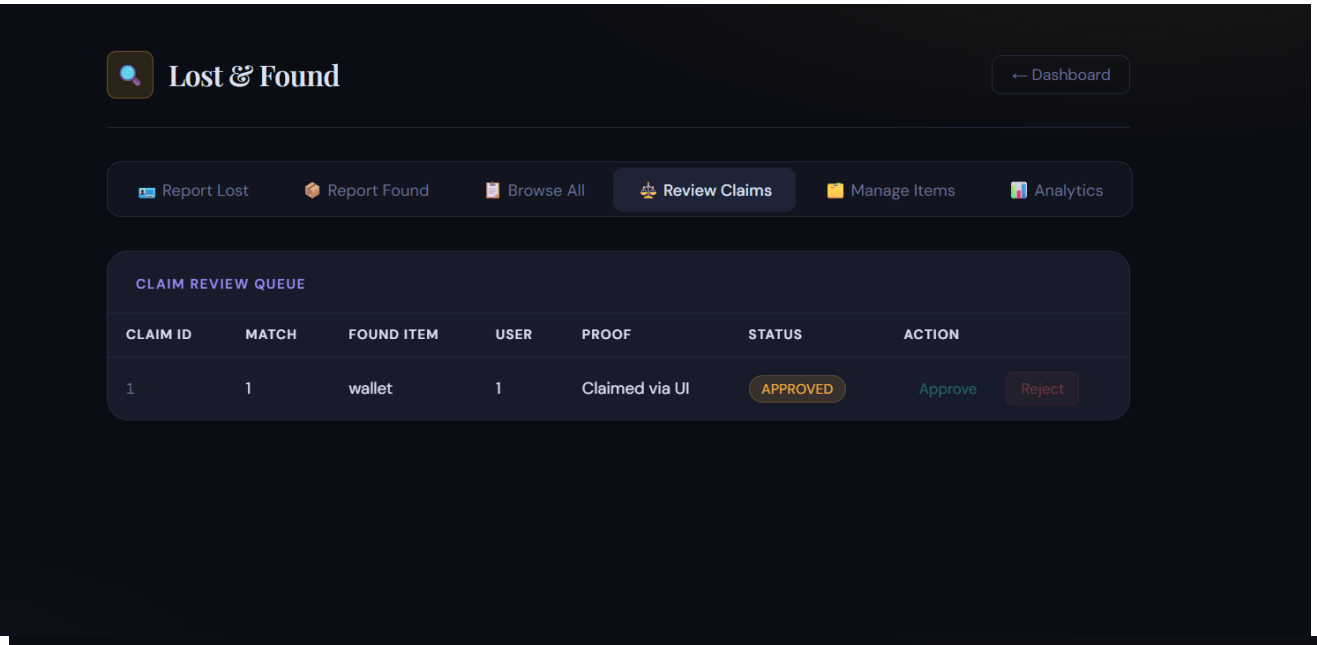

Lost & Found
← Dashboard
Report Lost
Report Found
Browse All
REPORT A LOST ITEM
ITEM NAME
wallet
YOUR NAME
anusha
LAST SEEN LOCATION
be block
DESCRIPTION
black colour
Submit Report

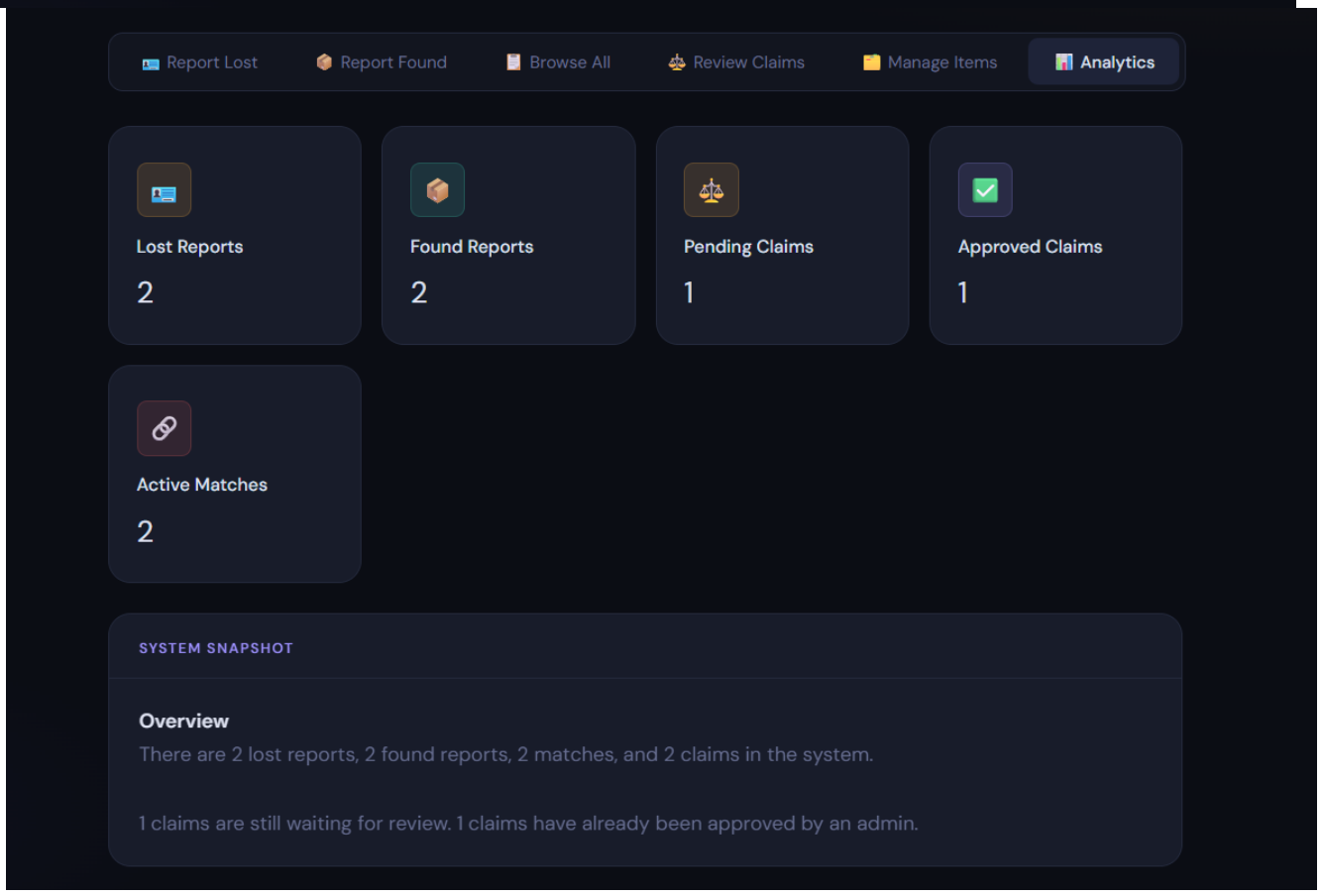

Lost & Found
← Dashboard
Report Lost
Report Found
Browse All
ALL LOST ITEMS
ID
NAME
DESCRIPTION
LOCATION
STATUS
REPORTED BY
1
wallet
black colour
be block
REPORTED
anusha
FOUND ITEMS
ALL FOUND ITEMS
ID
NAME
DESCRIPTION
LOCATION
ACTION
1
wallet
black colour
be block
Matched
Claim


FOUND ITEMS
ALL FOUND ITEMS
ID
NAME
DESCRIPTION
LOCATION
ACTION
1
wallet
black colour
be block
Claimed
Claimed ✓

Admin side-


DASHBOARD
ADMIN
Log out
Admin Dashboard
Manage, verify, and oversee all items.
QUICK ACTIONS
Verify Claims
Check proof of ownership for claims.
Approve / Reject
Finalise pending claim decisions.
Manage Items
Edit or remove items from the system.
View Analytics
Overview of activity and statistics.


Lost & Found
← Dashboard
Report Lost
Report Found
Browse All
Review Claims
Manage Items
Analytics
CLAIM REVIEW QUEUE
CLAIM ID
MATCH
FOUND ITEM
USER
PROOF
STATUS
ACTION
1
1
wallet
1
Claimed via UI
APPROVED
Approve
Reject


Report Lost
Report Found
Browse All
Review Claims
Manage Items
Analytics
Lost Reports
2
Found Reports
2
Pending Claims
1
Approved Claims
1
Active Matches
2
SYSTEM SNAPSHOT
Overview
There are 2 lost reports, 2 found reports, 2 matches, and 2 claims in the system.
1 claims are still waiting for review. 1 claims have already been approved by an admin.

Individual contributions of the team members:

Name	Module worked on
Amrutha S	Report lost item
Anurag sharma	Match items and notification
Anusha Balamurali	Report found item
Archi Pankaj	Claim and verification